

estack printable cheatsheet

One page map of what you can invoke, what euge-mode routes, and the principles behind the stack.

1. Start here

Use euge-mode for serious engineering work. It is the router.

2. Direct skills

Invoke a named workflow when you already know the shape you want.

3. Helpers

how, why, TypeScript, and unslop are narrow enough for model matching.

4. Principles

Treat principle-* files as routed references, not normal commands.

Invokable skills

commands and helpers

euge-mode

Default router for non-trivial work. Picks playbooks, principles, and helpers.

how

Explains subsystem structure, runtime flow, ownership, and layering.

why

Investigates history, rationale, tradeoffs, regressions, and thresholds.

architect

Sketches types, signatures, and module boundaries before implementation.

arena

Runs parallel attempts, chooses a base, and grafts in the best ideas.

interrogate

Adversarial multi-model review for a diff, design, or blind spots.

figure-it-out

Builds a custom auditable playbook when no bundled one fits.

reflect

Mines a completed task for durable lessons and routes skill edits.

automate-me

Creates or updates a personal mode skill from working style evidence.

show-me-your-work

Keeps a TSV decision trail for long or review-later work.

tdd

Writes a failing regression test first when the test path is cheap.

typescript-best-practices

TypeScript rules grounded in type-system discipline.

unslop

Cleans writing and removes AI tells.

euge-mode playbooks

router paths

investigation

Read-only question: how, why, are we sure, should we do X.

bug fix

Reproduce, root-cause, fix, and verify a defect.

perf issue

Trace measured slowness and improve against a baseline.

runtime forensics

Diagnose a live runtime symptom from instrumentation.

trace forensics

Diagnose a captured profile, trace, spindump, or heap snapshot.

feature

Build new or changed behavior from a named data shape.

refactoring

Preserve behavior while changing structure.

prototype

Build a cheap sketch to decide direction.

visual parity

Match UI pixels or styling between versions.

authoring a skill

Write or edit a SKILL.md.

eval

Test how a skill or prompt change affects behavior.

autonomous run

Drive a long task without stopping.

session pickup

Resume prior work from transcript, branch, or URL.

multi-phase plan

Plan work across phases or stacked PRs.

opening a PR

Publication workflow after other playbooks.

Principles

routed references

Laziness Protocol

Smallest change that solves the problem.

Foundational Thinking

Get data shape and shared ownership right first.

Redesign From First Principles

Integrate requirements as if known from day one.

Subtract Before You Add

Remove dead weight before building more.

Minimize Reader Load

Collapse needless layers and hidden state.

Outcome-Oriented Execution

Optimize for the verified end state.

Experience First

Choose user experience over convenience.

Exhaust the Design Space

Compare concrete options before hard choices.

Build the Lever

Script or generate repetitive work.

Boundary Discipline

Validate at boundaries; trust typed internals.

Type System Discipline

Make illegal states unrepresentable.

Make Operations Idempotent

Reruns and retries converge safely.

Migrate Callers Then Delete Legacy APIs

Move callers and remove old APIs together.

Separate Before Serializing Shared State

Eliminate shared writes before locks.

Prove It Works

Verify the real artifact, not a proxy.

Fix Root Causes

Reproduce, trace why, and fix the cause.

Guard the Context Window

Delegate bulk; keep summaries tight.

Never Block on the Human

Proceed on reversible work.

Encode Lessons in Structure

Turn repeated instructions into checks or tools.